Nama : **Rizal Endra Prayoga**

NPM : **06.2024.1.07753**

Jurusan : **Teknik Informatika**

Link Github : *https://github.com/christian1403/optek-basdat-inventarisku*

Analisis Optimasi Query untuk Menampilkan Riwayat Aktivitas Terakhir dan Total Biaya Pemeliharaan Aset Laptop Tahun 2023

1. Lingkungan Pengujian dan Dataset

Pengujian optimasi basis data dilakukan menggunakan PostgreSQL pada sistem inventaris aset. Struktur basis data terdiri atas tabel users, karyawan, status, lokasi, kategori, jenis_history, sparepart, teknisi, asset, dan asset_history.

Dataset dibuat menggunakan skrip SQL seed.sql yang memanfaatkan generate_series(), data master, dan proses pembangkitan data sintetis. Skrip menetapkan target sekitar 250.000 baris pada tabel asset dan sekitar 1.290.000 baris pada tabel asset_history. Jumlah aktual setiap tabel harus diverifikasi setelah proses seeding selesai.

Pengujian difokuskan pada query kompleks yang mencari aset laptop berdasarkan kategori dan tanggal pembelian, kemudian menggabungkan data lokasi dan riwayat aktivitas untuk menghasilkan informasi agregat serta aktivitas terbaru.

Lingkungan pengujian yang digunakan:

- RDBMS: PostgreSQL.
- Versi PostgreSQL: postgresql-18.6.
- Dataset: sistem inventaris aset.
- Query yang diuji: query laporan riwayat aktivitas dan biaya pemeliharaan aset laptop tahun 2023.
- Metode evaluasi: EXPLAIN (ANALYZE, BUFFERS) sebelum dan sesudah optimasi.

Penggunaan dataset sintetis berskala besar ditujukan untuk mengevaluasi metode akses data, mekanisme JOIN, proses agregasi, dan biaya eksekusi query dalam kondisi yang lebih realistis daripada pengujian menggunakan dataset kecil.

2. Querry BaseLine:

Query baseline digunakan untuk menampilkan informasi riwayat aktivitas dan total biaya pemeliharaan aset laptop yang dibeli selama tahun 2023. Query mengambil data dari tabel asset, menghubungkannya dengan tabel lokasi dan asset_history, serta menggunakan LEFT JOIN LATERAL untuk memperoleh jenis aktivitas terbaru dari setiap aset berdasarkan tanggal dan ID riwayat.

Informasi yang dihasilkan meliputi identitas aset, nama dan lokasi aset, jumlah riwayat aktivitas, jumlah jenis aktivitas yang berbeda, jumlah teknisi yang terlibat, total biaya riwayat, tanggal aktivitas terakhir, dan jenis aktivitas terakhir.

Query baseline dijalankan sebelum penambahan indeks optimasi, selain indeks bawaan yang sudah tersedia seperti indeks primary key dan indeks unik. Hasil eksekusi baseline digunakan untuk mengidentifikasi operasi yang paling banyak mengonsumsi waktu dan sumber daya, sehingga strategi optimasi dapat ditentukan berdasarkan bukti dari execution plan.

Querry Baseline :

```
SELECT
    a.id AS id_aset,
    a.nama_asset,
    a.kode_asset,
    a.brand,
    a.model,
    a.serial_number,
    a.tanggal_pembelian,
    l.nama AS lokasi,

    COUNT(h.id) AS jumlah_riwayat,
    COUNT(DISTINCT h.id_jenis) AS jumlah_jenis_aktivitas,
    COUNT(DISTINCT h.id_teknisi) AS jumlah_teknisi,

    SUM(COALESCE(h.biaya, 0)) AS total_biaya_riwayat,
    MAX(h.tanggal) AS tanggal_aktivitas_terakhir,

    MAX(terakhir.aktivitas_terakhir)
        AS jenis_aktivitas_terakhir

FROM public.asset a

LEFT JOIN public.lokasi l
    ON l.id = a.id_lokasi

LEFT JOIN public.asset_history h
    ON h.id_asset = a.id

LEFT JOIN LATERAL (
    SELECT
        j.nama AS aktivitas_terakhir
```

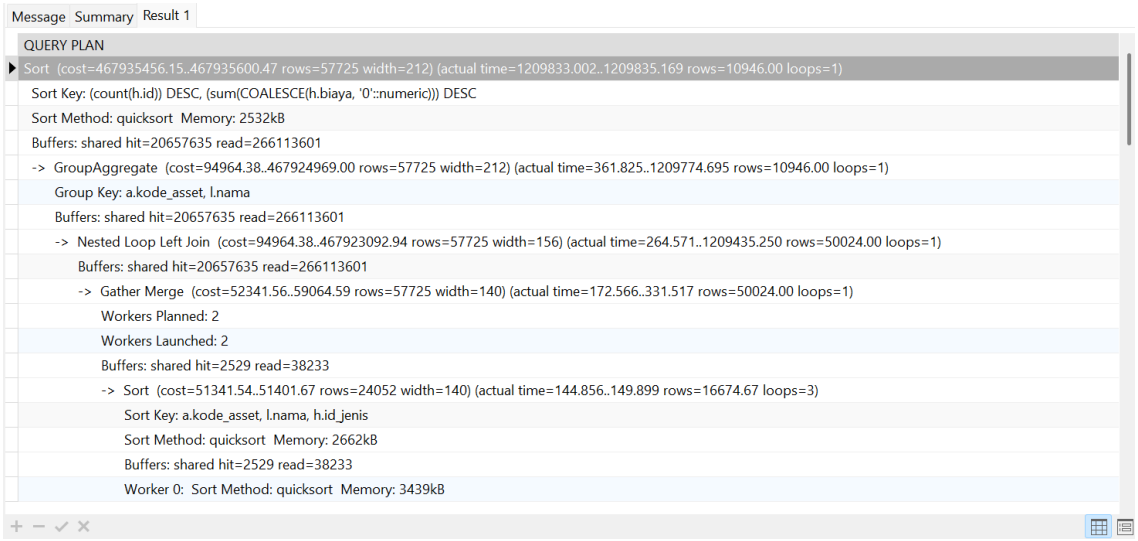
```
    FROM public.asset_history hx
    LEFT JOIN public.jenis_history j
        ON j.id = hx.id_jenis
    WHERE hx.id_asset = a.id
    ORDER BY hx.tanggal DESC, hx.id DESC
    LIMIT 1
) terakhir ON TRUE

WHERE a.id_kategori = 2
  AND a.tanggal_pembelian >= DATE '2023-01-01'
  AND a.tanggal_pembelian < DATE '2024-01-01'

GROUP BY
    a.id,
    a.nama_asset,
    a.kode_asset,
    a.brand,
    a.model,
    a.serial_number,
    a.tanggal_pembelian,
    l.nama

ORDER BY
    jumlah_riwayat DESC,
    total_biaya_riwayat DESC
```

Hasil Explain(ANALYZE,BUFFERS)



Message | Summary | Result 1

QUERY PLAN

Sort (cost=467935456.15..467935600.47 rows=57725 width=212) (actual time=1209833.002..1209835.169 rows=10946.00 loops=1)
Sort Key: (count(h.id)) DESC, (sum(COALESCE(h.biaya, '0'::numeric))) DESC
Sort Method: quicksort Memory: 2532kB
Buffers: shared hit=20657635 read=266113601
-> GroupAggregate (cost=94964.38..467924969.00 rows=57725 width=212) (actual time=361.825..1209774.695 rows=10946.00 loops=1)
Group Key: a.kode_asset, l.nama
Buffers: shared hit=20657635 read=266113601
-> Nested Loop Left Join (cost=94964.38..467923092.94 rows=57725 width=156) (actual time=264.571..1209435.250 rows=50024.00 loops=1)
Buffers: shared hit=20657635 read=266113601
-> Gather Merge (cost=52341.56..59064.59 rows=57725 width=140) (actual time=172.566..331.517 rows=50024.00 loops=1)
Workers Planned: 2
Workers Launched: 2
Buffers: shared hit=2529 read=38233
-> Sort (cost=51341.54..51401.67 rows=24052 width=140) (actual time=144.856..149.899 rows=16674.67 loops=3)
Sort Key: a.kode_asset, l.nama, h.id_jenis
Sort Method: quicksort Memory: 2662kB
Buffers: shared hit=2529 read=38233
Worker 0: Sort Method: quicksort Memory: 3439kB

Message Summary Result 1

```
QUERY PLAN
                  Worker 1:  Sort Method: quicksort  Memory: 3566kB
                  ->  Hash Left Join  (cost=16415.19..49591.29 rows=24052 width=140) (actual time=49.209..105.472 rows=16674.67 loops=3)
                        Hash Cond: (a.id_lokasi = l.id)
                        Buffers: shared hit=2499 read=38233
                        ->  Parallel Hash Right Join  (cost=16413.07..49520.68 rows=24052 width=112) (actual time=48.692..102.722 rows=16674.67 loops=3)
                              Hash Cond: (h.id_asset = a.id)
                              Buffers: shared hit=2496 read=38233
                              ->  Parallel Seq Scan on asset_history h  (cost=0.00..31670.32 rows=547532 width=24) (actual time=0.714..28.194 rows=438025.67 loops=3)
                                    Buffers: shared hit=1605 read=24590
                              ->  Parallel Hash  (cost=16355.90..16355.90 rows=4573 width=92) (actual time=27.115..27.117 rows=3648.67 loops=3)
                                    Buckets: 16384  Batches: 1  Memory Usage: 1600kB
                                    Buffers: shared hit=891 read=13643
                                    ->  Parallel Seq Scan on asset a  (cost=0.00..16355.90 rows=4573 width=92) (actual time=6.675..25.993 rows=3648.67 loops=3)
                                          Filter: ((tanggal_pembelian >= '2023-01-01'::date) AND (tanggal_pembelian < '2023-12-31'::date) AND (id_kategori = 2))
                                          Rows Removed by Filter: 79638
                                          Buffers: shared hit=891 read=13643
                        ->  Hash  (cost=1.50..1.50 rows=50 width=36) (actual time=0.499..0.500 rows=50.00 loops=3)
                              Buckets: 1024  Batches: 1  Memory Usage: 12kB
```

Message Summary Result 1

```
QUERY PLAN
                              Buffers: shared hit=3
                              ->  Seq Scan on lokasi l  (cost=0.00..1.50 rows=50 width=36) (actual time=0.485..0.488 rows=50.00 loops=3)
                                    Buffers: shared hit=3
            ->  Memoize  (cost=42622.82..42622.83 rows=1 width=16) (actual time=24.169..24.169 rows=1.00 loops=50024)
                  Cache Key: a.id
                  Cache Mode: binary
                  Hits: 39078  Misses: 10946  Evictions: 0  Overflows: 0  Memory Usage: 1232kB
                  Buffers: shared hit=20655106 read=266075368
                  ->  Subquery Scan on terakhir  (cost=42622.81..42622.82 rows=1 width=16) (actual time=110.449..110.450 rows=1.00 loops=10946)
                        Buffers: shared hit=20655106 read=266075368
                        ->  Limit  (cost=42622.81..42622.81 rows=1 width=24) (actual time=110.447..110.447 rows=1.00 loops=10946)
                              Buffers: shared hit=20655106 read=266075368
                              ->  Sort  (cost=42622.81..42622.82 rows=6 width=24) (actual time=110.445..110.445 rows=1.00 loops=10946)
                                    Sort Key: hx.tanggal DESC, hx.id DESC
                                    Sort Method: top-N heapsort  Memory: 25kB
                                    Buffers: shared hit=20655106 read=266075368
                                    ->  Nested Loop Left Join  (cost=0.00..42622.78 rows=6 width=24) (actual time=35.572..110.423 rows=4.57 loops=10946)
                                          Join Filter: (j.id = hx.id_jenis)
```

Message Summary Result 1

```
QUERY PLAN
                                    Buffers: shared hit=20655106 read=266075368
                                    ->  Nested Loop Left Join  (cost=0.00..42622.78 rows=6 width=24) (actual time=35.572..110.423 rows=4.57 loops=10946)
                                          Join Filter: (j.id = hx.id_jenis)
                                          Rows Removed by Join Filter: 12
                                          Buffers: shared hit=20655103 read=266075368
                                          ->  Seq Scan on asset_history hx  (cost=0.00..42620.96 rows=6 width=12) (actual time=35.547..110.378 rows=4.57 loops=10946)
                                                Filter: (id_asset = a.id)
                                                Rows Removed by Filter: 1314072
                                                Buffers: shared hit=20655102 read=266075368
                                          ->  Materialize  (cost=0.00..1.14 rows=9 width=20) (actual time=0.001..0.001 rows=3.66 loops=50024)
                                                Storage: Memory  Maximum Storage: 17kB
                                                Buffers: shared hit=1
                                                ->  Seq Scan on jenis_history j  (cost=0.00..1.09 rows=9 width=20) (actual time=0.020..0.026 rows=9.00 loops=1)
                                                      Buffers: shared hit=1
Planning:
  Buffers: shared hit=24 dirtied=1
Planning Time: 1.299 ms
Execution Time: 1209836.843 ms
```

Penyebab bottleneck pada querry baseline:

1. Sequential Scan pada asset_history

```
-> Seq Scan on asset_history hx  (cost=0.00..42620.96 rows=6 width=12) (actual time=35.547..110.378 rows=4.57 loops=10946)
   Filter: (id_asset = a.id)
   Rows Removed by Filter: 1314072
   Buffers: shared hit=20655102 read=266075368
```

   Pada hasil execution plan ditemukan operasi Sequential Scan pada tabel asset_history dengan jumlah perulangan sebanyak 10.946 kali. Setiap pemindaian memeriksa sekitar 1,31 juta baris, sedangkan rata-rata hanya sekitar 4,57 baris yang memenuhi kondisi pencarian. Hal ini menunjukkan terjadinya pembacaan data berulang dalam jumlah besar sehingga operasi tersebut menjadi bottleneck utama query.

2. Nested Loop Left Join

```
-> Nested Loop Left Join  (cost=94964.38..467923092.94 rows=57725 width=156) (actual time=264.571..1209435.250 rows=50024.00 loops=1)
   Buffers: shared hit=20657635 read=266113601
```

   Operasi Nested Loop Left Join menunjukkan waktu aktual hingga 1.209.435,250 ms. Tingginya durasi operasi ini disebabkan oleh subquery aktivitas terakhir yang dijalankan untuk setiap kunci aset yang diperlukan. Dengan demikian, Nested Loop meningkat karena proses pencarian riwayat di dalamnya belum menggunakan indeks yang sesuai.

3. Memoize dan Subquery Scan

```
-> Memoize  (cost=42622.82..42622.83 rows=1 width=16) (actual time=24.169..24.169 rows=1.00 loops=50024)
   Cache Key: a.id
   Cache Mode: binary
   Hits: 39078  Misses: 10946  Evictions: 0  Overflows: 0  Memory Usage: 1232kB
   Buffers: shared hit=20655106 read=266075368
   -> Subquery Scan on terakhir  (cost=42622.81..42622.82 rows=1 width=16) (actual time=110.449..110.450 rows=1.00 loops=10946)
```

   Operator Memoize berhasil menyimpan hasil pencarian sehingga terdapat 39.078 cache hit. Namun, masih terdapat 10.946 cache miss yang mengharuskan PostgreSQL menjalankan subquery aktivitas terakhir. Setiap cache miss tersebut memicu pemindaian tabel asset_history. Oleh karena itu, walaupun Memoize membantu mengurangi pengulangan pencarian, bottleneck belum dapat diatasi tanpa perbaikan metode akses data.

4. Parallel Sequential Scan pada tabel asset

```
Parallel Seq Scan on asset a  (cost=0.00..16355.90 rows=4573 width=92) (actual time=6.675..25.993 rows=3648.67 loops=3)
Filter: ((tanggal_pembelian >= '2023-01-01'::date) AND (tanggal_pembelian < '2023-12-31'::date) AND (id_kategori = 2))
Rows Removed by Filter: 79638
Buffers: shared hit=891 read=13643
```

Pada tabel asset ditemukan Parallel Sequential Scan dengan filter kategori dan tanggal pembelian. Sebagian besar baris tidak memenuhi kondisi filter. Meskipun demikian, waktu aktual pemindaian masih relatif rendah dibandingkan dengan subquery pencarian riwayat, sehingga bagian ini tidak diprioritaskan sebagai bottleneck utama.

5. Sort dan GroupAggregate

```
Sort  (cost=467935456.15..467935600.47 rows=57725 width=212) (actual time=1209833.002..1209835.169 rows=10946.00 loops=1)
  Sort Key: (count(h.id)) DESC, (sum(COALESCE(h.biaya, '0'::numeric))) DESC
  Sort Method: quicksort  Memory: 2532kB
  Buffers: shared hit=20657635 read=266113601
  ->  GroupAggregate  (cost=94964.38..467924969.00 rows=57725 width=212) (actual time=361.825..1209774.695 rows=10946.00 loops=1)
```

Operasi Sort dan GroupAggregate memiliki waktu aktual yang tinggi karena keduanya berada di atas operasi JOIN dan subquery yang mahal. Namun, metode pengurutan yang digunakan adalah quicksort dengan penggunaan memori sekitar 2.532 kB. Oleh karena itu, tingginya waktu pada node Sort dan GroupAggregate terutama berkaitan dengan lamanya proses menghasilkan data dari node sebelumnya, bukan karena pengurutan menjadi bottleneck utama.

3. Rencana Optimasi Basis Data

Berdasarkan hasil analisis execution plan baseline, bottleneck utama ditemukan pada pencarian aktivitas terakhir di tabel asset_history. PostgreSQL melakukan Sequential Scan secara berulang untuk mencari riwayat berdasarkan id_asset, kemudian mengurutkan hasil berdasarkan tanggal dan ID riwayat.

Strategi optimasi yang dipilih adalah penerapan Composite B-Tree Index pada tabel asset_history. Indeks ini menggunakan tiga kolom dengan urutan id_asset, tanggal DESC, dan id DESC.

Kolom id_asset digunakan untuk menemukan riwayat milik aset tertentu. Kolom tanggal DESC mendukung pengambilan aktivitas berdasarkan tanggal terbaru, sedangkan kolom id DESC menjadi kriteria pengurutan tambahan apabila beberapa aktivitas memiliki tanggal yang sama.

Indeks dirancang sesuai pola akses pada subquery LEFT JOIN LATERAL, yang menggunakan kondisi pencarian berdasarkan aset, pengurutan menurun, dan LIMIT 1. Dengan demikian, PostgreSQL berpeluang mengambil riwayat terbaru melalui struktur indeks tanpa harus memindai seluruh tabel pada setiap pencarian.

Strategi ini memprioritaskan bottleneck yang teridentifikasi dalam execution plan, Efektivitas indeks tetap harus dibuktikan melalui perbandingan execution plan dan waktu eksekusi aktual.

Strategi Optimasi

Strategi optimasi yang dipilih adalah penerapan -Index pada tabel asset_history. Pemilihan indeks ini didasarkan pada bagian query yang digunakan untuk mencari aktivitas terakhir setiap aset:

```
SELECT j.nama AS aktivitas_terakhir
FROM public.asset_history hx
LEFT JOIN public.jenis_history j
    ON j.id = hx.id_jenis
WHERE hx.id_asset = a.id
ORDER BY hx.tanggal DESC, hx.id DESC
LIMIT 1
```

Pada query tersebut, terdapat tiga kolom yang berperan penting:

- id_asset, digunakan sebagai kondisi pencarian riwayat berdasarkan identitas aset.
- tanggal, digunakan untuk mengurutkan aktivitas berdasarkan tanggal terbaru.
- id, digunakan sebagai pengurutan tambahan apabila terdapat beberapa aktivitas dengan tanggal yang sama.

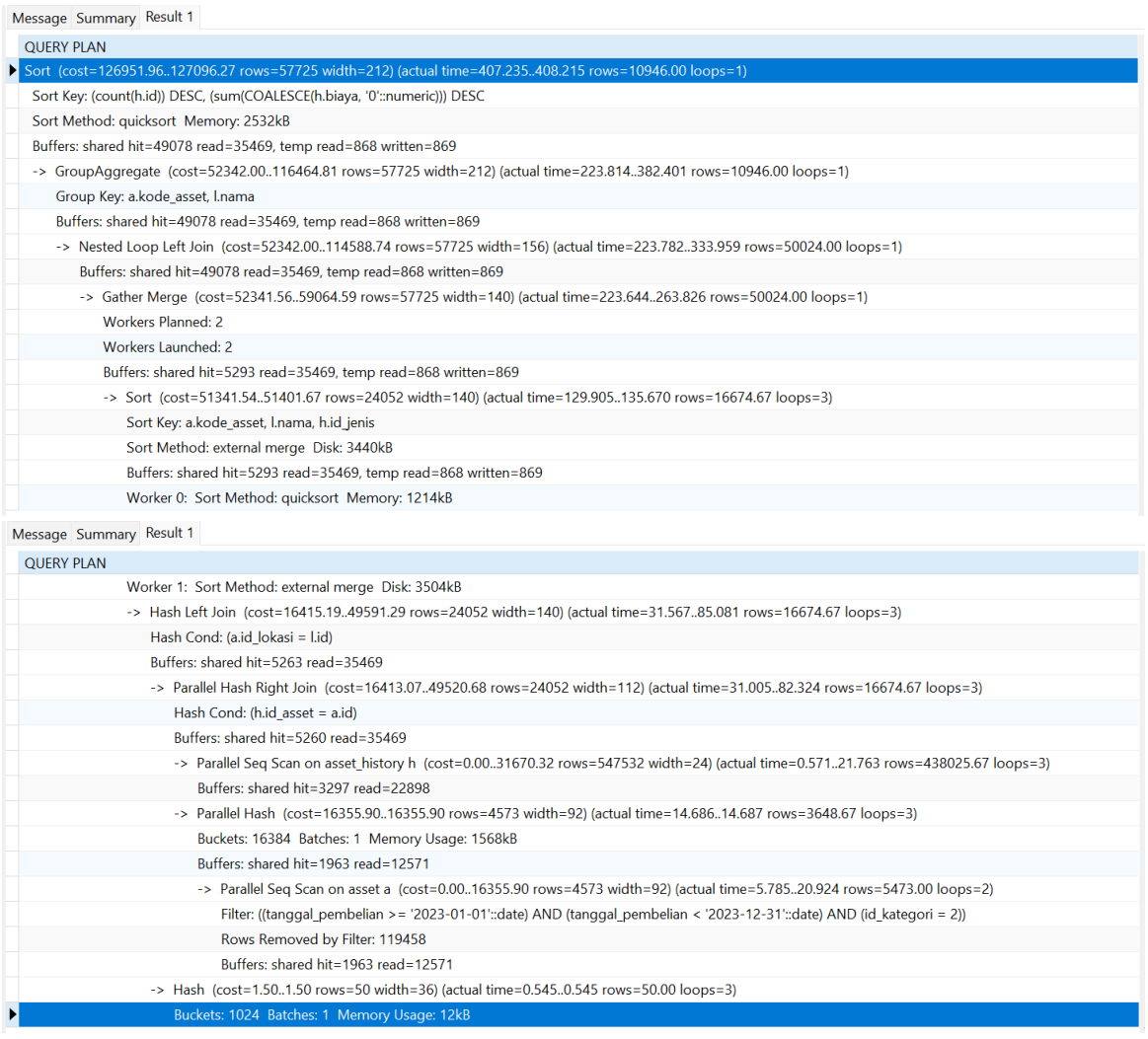
Index yang akan di buat untuk mengatasi bottleneck:

```
CREATE INDEX idx_history_asset_latest
ON public.asset_history (
    id_asset,
    tanggal DESC,
    id DESC
);
```

4. Hasil Optimasi Basis Data

Setelah dilakukan analisis terhadap bottleneck pada query baseline, tahap selanjutnya adalah menerapkan Composite B-Tree Index pada tabel asset_history.

Indeks dibuat untuk mempercepat pencarian aktivitas terakhir berdasarkan id_asset serta mendukung pengurutan berdasarkan tanggal DESC dan id DESC.



Message Summary Result 1

```
QUERY PLAN
Sort  (cost=126951.96..127096.27 rows=57725 width=212) (actual time=407.235..408.215 rows=10946.00 loops=1)
  Sort Key: (count(h.id)) DESC, (sum(COALESCE(h.biaya, '0'::numeric))) DESC
  Sort Method: quicksort  Memory: 2532kB
  Buffers: shared hit=49078 read=35469, temp read=868 written=869
  ->  GroupAggregate  (cost=52342.00..116464.81 rows=57725 width=212) (actual time=223.814..382.401 rows=10946.00 loops=1)
        Group Key: a.kode_asset, l.nama
        Buffers: shared hit=49078 read=35469, temp read=868 written=869
        ->  Nested Loop Left Join  (cost=52342.00..114588.74 rows=57725 width=156) (actual time=223.782..333.959 rows=50024.00 loops=1)
              Buffers: shared hit=49078 read=35469, temp read=868 written=869
              ->  Gather Merge  (cost=52341.56..59064.59 rows=57725 width=140) (actual time=223.644..263.826 rows=50024.00 loops=1)
                    Workers Planned: 2
                    Workers Launched: 2
                    Buffers: shared hit=5293 read=35469, temp read=868 written=869
                    ->  Sort  (cost=51341.54..51401.67 rows=24052 width=140) (actual time=129.905..135.670 rows=16674.67 loops=3)
                          Sort Key: a.kode_asset, l.nama, h.id_jenis
                          Sort Method: external merge  Disk: 3440kB
                          Buffers: shared hit=5293 read=35469, temp read=868 written=869
                          Worker 0:  Sort Method: quicksort  Memory: 1214kB
```

Message Summary Result 1

```
QUERY PLAN
                    Worker 1:  Sort Method: external merge  Disk: 3504kB
                    ->  Hash Left Join  (cost=16415.19..49591.29 rows=24052 width=140) (actual time=31.567..85.081 rows=16674.67 loops=3)
                          Hash Cond: (a.id_lokasi = l.id)
                          Buffers: shared hit=5263 read=35469
                          ->  Parallel Hash Right Join  (cost=16413.07..49520.68 rows=24052 width=112) (actual time=31.005..82.324 rows=16674.67 loops=3)
                                Hash Cond: (h.id_asset = a.id)
                                Buffers: shared hit=5260 read=35469
                                ->  Parallel Seq Scan on asset_history h  (cost=0.00..31670.32 rows=547532 width=24) (actual time=0.571..21.763 rows=438025.67 loops=3)
                                      Buffers: shared hit=3297 read=22898
                                ->  Parallel Hash  (cost=16355.90..16355.90 rows=4573 width=92) (actual time=14.686..14.687 rows=3648.67 loops=3)
                                      Buckets: 16384  Batches: 1  Memory Usage: 1568kB
                                      Buffers: shared hit=1963 read=12571
                                      ->  Parallel Seq Scan on asset a  (cost=0.00..16355.90 rows=4573 width=92) (actual time=5.785..20.924 rows=5473.00 loops=2)
                                            Filter: ((tanggal_pembelian >= '2023-01-01'::date) AND (tanggal_pembelian < '2023-12-31'::date) AND (id_kategori = 2))
                                            Rows Removed by Filter: 119458
                                            Buffers: shared hit=1963 read=12571
                          ->  Hash  (cost=1.50..1.50 rows=50 width=36) (actual time=0.545..0.545 rows=50.00 loops=3)
                                Buckets: 1024  Batches: 1  Memory Usage: 12kB
```

Message Summary Result 1

```
QUERY PLAN
                        Buffers: shared hit=3
                        -> Seq Scan on lokasi l  (cost=0.00..1.50 rows=50 width=36) (actual time=0.528..0.531 rows=50.00 loops=3)
                            Buffers: shared hit=3
    -> Memoize  (cost=0.44..4.94 rows=1 width=16) (actual time=0.001..0.001 rows=1.00 loops=50024)
        Cache Key: a.id
        Cache Mode: binary
        Hits: 39078  Misses: 10946  Evictions: 0  Overflows: 0  Memory Usage: 1232kB
        Buffers: shared hit=43785
        -> Subquery Scan on terakhir  (cost=0.43..4.93 rows=1 width=16) (actual time=0.003..0.004 rows=1.00 loops=10946)
            Buffers: shared hit=43785
            -> Limit  (cost=0.43..4.92 rows=1 width=24) (actual time=0.003..0.003 rows=1.00 loops=10946)
                Buffers: shared hit=43785
                -> Nested Loop Left Join  (cost=0.43..27.36 rows=6 width=24) (actual time=0.003..0.003 rows=1.00 loops=10946)
                    Join Filter: (j.id = hx.id_jenis)
                    Rows Removed by Join Filter: 3
                    Buffers: shared hit=43785
                    -> Index Scan using idx_history_asset_latest on asset_history hx  (cost=0.43..25.55 rows=6 width=12) (actual time=0.002..0.002 rows=1.00 loops=10946)
                        Index Cond: (id_asset = a.id)
```

Message Summary Result 1

```
QUERY PLAN
                Buffers: shared hit=43785
                -> Nested Loop Left Join  (cost=0.43..27.36 rows=6 width=24) (actual time=0.003..0.003 rows=1.00 loops=10946)
                    Join Filter: (j.id = hx.id_jenis)
                    Rows Removed by Join Filter: 3
                    Buffers: shared hit=43785
                    -> Index Scan using idx_history_asset_latest on asset_history hx  (cost=0.43..25.55 rows=6 width=12) (actual time=0.002..0.002 rows=1.00 loops=10946)
                        Index Cond: (id_asset = a.id)
                        Index Searches: 10946
                        Buffers: shared hit=43784
                    -> Materialize  (cost=0.00..1.14 rows=9 width=20) (actual time=0.000..0.000 rows=4.30 loops=10946)
                        Storage: Memory  Maximum Storage: 17kB
                        Buffers: shared hit=1
                        -> Seq Scan on jenis_history j  (cost=0.00..1.09 rows=9 width=20) (actual time=0.013..0.014 rows=9.00 loops=1)
                            Buffers: shared hit=1
Planning:
  Buffers: shared hit=24
Planning Time: 0.362 ms
Execution Time: 418.001 ms
```

1. Perubahan Scan pada asset_history

   Sebelum:

```
-> Seq Scan on asset_history hx  (cost=0.00..42620.96 rows=6 width=12) (actual time=35.547..110.378 rows=4.57 loops=10946)
    Filter: (id_asset = a.id)
    Rows Removed by Filter: 1314072
    Buffers: shared hit=20655102 read=266075368
```

   Sesudah:

```
-> Index Scan using idx_history_asset_latest on asset_history hx  (cost=0.43..25.55 rows=6 width=12) (actual time=0.002..0.002 rows=1.00 loops=10946)
    Index Cond: (id_asset = a.id)
    Index Searches: 10946
    Buffers: shared hit=43784
```

   Sebelum indeks diterapkan, PostgreSQL melakukan Sequential Scan sebanyak 10.946 kali untuk mencari aktivitas terakhir dari setiap aset. Setelah indeks diterapkan, pencarian tetap dilakukan sebanyak 10.946 kali, tetapi sudah menggunakan Index Scan. Pada kondisi setelah optimasi, PostgreSQL dapat menemukan aktivitas terbaru secara langsung berdasarkan struktur indeks, sehingga tidak perlu memindai seluruh tabel

asset_history pada setiap pencarian. Perubahan ini membuktikan bahwa indeks yang dibuat sesuai dengan pola pencarian dan pengurutan pada query.

2. Perubahan pada Operasi Memoize

   Sebelum :

```
-> Memoize  (cost=42622.82..42622.83 rows=1 width=16) (actual time=24.169..24.169 rows=1.00 loops=50024)
   Cache Key: a.id
   Cache Mode: binary
   Hits: 39078  Misses: 10946  Evictions: 0  Overflows: 0  Memory Usage: 1232kB
   Buffers: shared hit=20655106 read=266075368
   -> Subquery Scan on terakhir  (cost=42622.81..42622.82 rows=1 width=16) (actual time=110.449..110.450 rows=1.00 loops=10946)
```

   Sesudah :

```
-> Memoize  (cost=0.44..4.94 rows=1 width=16) (actual time=0.001..0.001 rows=1.00 loops=50024)
   Cache Key: a.id
   Cache Mode: binary
   Hits: 39078  Misses: 10946  Evictions: 0  Overflows: 0  Memory Usage: 1232kB
   Buffers: shared hit=43785
```

   Jumlah cache hit dan cache miss tidak mengalami perubahan dibandingkan dengan kondisi sebelum optimasi. Namun, terdapat perubahan pada operasi yang dijalankan ketika terjadi cache miss. Sebelum optimasi, cache miss menyebabkan PostgreSQL melakukan Sequential Scan pada tabel asset_history. Setelah optimasi, cache miss menyebabkan PostgreSQL menggunakan Index Scan untuk mengambil riwayat aset. Dengan demikian, meskipun jumlah pencarian tidak berkurang, waktu yang diperlukan untuk menyelesaikan pencarian menjadi jauh lebih singkat.

3. Perubahan Nested Loop Left Join

   Sebelum :

```
-> Nested Loop Left Join  (cost=94964.38..467923092.94 rows=57725 width=156) (actual time=264.571..1209435.250 rows=50024.00 loops=1)
   Buffers: shared hit=20657635 read=266113601
```

   Sesudah :

```
-> Nested Loop Left Join  (cost=52342.00..114588.74 rows=57725 width=156) (actual time=223.782..333.959 rows=50024.00 loops=1)
   Buffers: shared hit=49078 read=35469, temp read=868 written=869
```

   Sebelum indeks diterapkan, operasi Nested Loop membutuhkan waktu hingga sekitar 1.209.435 ms. Setelah indeks diterapkan, waktu pada node tersebut berakhir sekitar 334 ms. Penurunan ini terjadi karena subquery aktivitas terakhir yang berada di dalam Nested Loop kini menggunakan indeks untuk mencari data. Hal ini menunjukkan bahwa optimasi tidak harus mengubah algoritma JOIN menjadi algoritma lain. Perbaikan

metode akses data pada operasi di dalam JOIN juga dapat meningkatkan performa query secara signifikan.

4. Perubahan Estimated Cost

   Berdasarkan execution plan, terdapat perubahan estimated total cost sebagai berikut:

Kondisi	Estimated Total Cost
Sebelum Optimasi	467.935.600,47
Setelah Optimasi	127.096,27

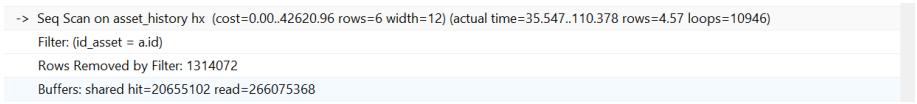
   Penurunan estimated cost menunjukkan bahwa PostgreSQL memperkirakan penggunaan indeks lebih efisien dibandingkan dengan pemindaian tabel berulang.

   Estimated cost merupakan nilai perhitungan query planner, sehingga tidak secara langsung menunjukkan waktu eksekusi aktual.

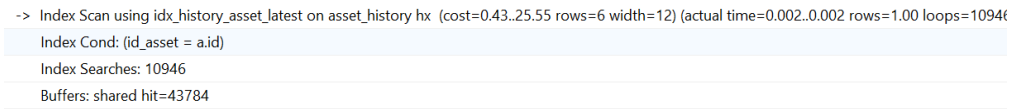
   Namun, penurunan estimated cost tersebut sejalan dengan perubahan metode akses data dan hasil pengukuran execution time.

5. Analisis Penggunaan Buffer

   Sebelum optimasi, pada node pencarian riwayat ditemukan:



   Setelah optimasi, pada node Index Scan ditemukan:



   Perbandingan ini menunjukkan penurunan aktivitas akses buffer yang sangat besar pada proses pencarian aktivitas terakhir. Sebelum optimasi, Sequential Scan berulang menyebabkan pembacaan halaman data dalam jumlah besar. Setelah optimasi, PostgreSQL menggunakan indeks untuk menemukan riwayat yang sesuai sehingga jumlah akses buffer pada bagian tersebut jauh lebih rendah. Meskipun demikian, penggunaan buffer pada keseluruhan query masih dipengaruhi oleh operasi lainnya, seperti Parallel Sequential Scan, JOIN, dan Sort.

5. Tabel Perbandingan Hasil Optimasi

Parameter	Sebelum Optimasi	Sesudah Optimasi
Indeks optimasi	Belum Di terapkan	idx_history_asset_latest
Metode pencarian aktivitas terakhir	Sequential Scan berulang	Index Scan
Execution Time	1.209.836,843 ms	418,001 ms
Estimated Total Cost	467.935.600,47	127.096,27
Persentase penurunan waktu eksekusi	-	Sekitar 99,965%
Ukuran indeks optimasi	Belum tersedia	39 MB

Perbandingan menunjukkan perubahan metode akses data pada proses pencarian aktivitas terakhir. Sebelum optimasi, PostgreSQL melakukan Sequential Scan berulang pada tabel asset_history. Setelah indeks diterapkan, PostgreSQL menggunakan Index Scan untuk menemukan riwayat berdasarkan id_asset dan urutan tanggal serta ID.

Perubahan tersebut sejalan dengan penurunan estimated cost dan execution time. Namun, estimated cost merupakan estimasi biaya dari query planner, bukan waktu aktual. Ukuran indeks juga perlu dilaporkan untuk memperlihatkan kebutuhan penyimpanan yang timbul akibat optimasi.

6. Pengukuran Ukuran Indeks

```
SELECT
    indexrelname AS nama_indeks,
    pg_size_pretty(
        pg_relation_size(indexrelid)
    ) AS ukuran_indeks,
    pg_relation_size(indexrelid) AS ukuran_byte
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
  AND relname = 'asset_history'
ORDER BY pg_relation_size(indexrelid) DESC;
```

Message | Summary | Result 1

nama_indeks	ukuran_indeks	ukuran_byte
idx_history_asset_latest	39 MB	41410560
asset_history_pkey	28 MB	29540352

Pengukuran ukuran indeks dilakukan untuk mengevaluasi kebutuhan ruang penyimpanan yang timbul akibat penerapan strategi optimasi. PostgreSQL menyediakan fungsi pg_relation_size() untuk memperoleh ukuran aktual suatu

indeks dalam byte. Fungsi pg_size_pretty() digunakan untuk menyajikan ukuran tersebut dalam satuan yang lebih mudah dibaca.

Pengukuran dilakukan pada indeks idx_history_asset_latest yang diterapkan pada tabel asset_history. Hasil pengukuran dicatat untuk mengetahui besarnya ruang penyimpanan yang digunakan oleh indeks optimasi.

Ukuran indeks menjadi salah satu aspek evaluasi karena peningkatan kinerja query diperoleh dengan konsekuensi penambahan struktur penyimpanan. Oleh sebab itu, hasil pengukuran ukuran indeks dianalisis bersama perubahan execution time dan overhead operasi DML.

Hasil pengukuran:

- Nama indeks: idx_history_asset_latest.
- Ukuran indeks: 39 MB.
- Ukuran dalam byte: 41410560.

Nilai tersebut merupakan ukuran indeks pada saat pengukuran dilakukan. Nilai aktual dapat berbeda bergantung pada jumlah baris, distribusi data, dan kondisi fisik penyimpanan indeks.

7. Pengujian Overhead DML

Pengujian Insert :

```
ROLLBACK;

BEGIN;

EXPLAIN (ANALYZE, BUFFERS, WAL)
INSERT INTO public.asset_history
    (id_asset, tanggal, id_jenis, id_status, biaya)
SELECT
    id_asset,
    tanggal,
    id_jenis,
    id_status,
    biaya
FROM public.asset_history
ORDER BY id
LIMIT 1000;

ROLLBACK;
```

Pengujian Update :

```
ROLLBACK;

BEGIN;

EXPLAIN (ANALYZE, BUFFERS, WAL)
UPDATE public.asset_history
SET tanggal = tanggal + INTERVAL '1 day'
WHERE id IN (
    SELECT id
    FROM public.asset_history
    ORDER BY id
    LIMIT 1000
);

ROLLBACK;
```

Pengjuan Delete :

```
ROLLBACK;

BEGIN;

EXPLAIN (ANALYZE, BUFFERS, WAL)
DELETE FROM public.asset_history
WHERE id IN (
    SELECT id
    FROM public.asset_history
    ORDER BY id
    LIMIT 1000
);

ROLLBACK;
```

Operasi	Jumlah Baris	Waktu tanpa indeks Optimasi	Waktu dengan indeks optimasi
Insert	1.000	106,063 ms	47.494 ms
Update	Maks. 1.000	7.479 ms	7.062 ms
Delete	Maks. 1.000	2.486 ms	1.923 ms

Pengujian overhead DML dilakukan untuk membandingkan waktu eksekusi operasi INSERT, UPDATE, dan DELETE sebelum dan sesudah penerapan indeks optimasi pada tabel asset_history. Berdasarkan hasil pengukuran awal, operasi INSERT untuk 1.000 baris menunjukkan waktu eksekusi sebesar 106,063 ms tanpa indeks optimasi dan 47,494 ms dengan indeks optimasi. Operasi UPDATE menunjukkan waktu

sebesar 7,479 ms dan 7,062 ms, sedangkan operasi DELETE menunjukkan waktu sebesar 2,486 ms dan 1,923 ms.

Hasil pengukuran memperlihatkan penurunan waktu eksekusi pada ketiga operasi DML setelah indeks diterapkan. Namun, perbedaan waktu tersebut belum cukup untuk menyimpulkan bahwa indeks optimasi secara langsung meningkatkan kinerja seluruh operasi DML. Indeks yang digunakan terutama dirancang untuk mendukung pencarian riwayat aset berdasarkan id_asset serta pengurutan berdasarkan tanggal dan id. Oleh karena itu, manfaat utamanya perlu dievaluasi berdasarkan kesesuaiannya dengan pola akses data yang digunakan.

Secara teknis, penambahan indeks dapat meningkatkan performa penulisan karena PostgreSQL harus memelihara struktur indeks ketika data mengalami perubahan. Perbedaan waktu yang terukur juga dapat dipengaruhi oleh kondisi cache, aktivitas server, dan variasi eksekusi. Dengan demikian, hasil awal ini perlu diverifikasi melalui pengujian berulang menggunakan query, jumlah baris, serta kondisi pengujian yang konsisten

8. Kesimpulan Hasil Optimasi

Berdasarkan hasil pengujian query laporan riwayat aktivitas dan total biaya pemeliharaan aset laptop tahun 2023, bottleneck utama pada kondisi baseline berasal dari pencarian aktivitas terakhir pada tabel asset_history. PostgreSQL melakukan Sequential Scan secara berulang untuk menemukan riwayat berdasarkan id_asset, sehingga proses tersebut membutuhkan waktu eksekusi yang tinggi.

Penerapan Composite B-Tree Index pada kolom (id_asset, tanggal DESC, id DESC) mengubah metode akses data pada pencarian aktivitas terakhir menjadi Index Scan. Berdasarkan hasil yang dicatat dalam pengujian, waktu eksekusi berkurang dari 1.209.836,843 ms menjadi 418,001 ms. Perubahan ini menunjukkan bahwa struktur indeks yang sesuai dengan kondisi pencarian dan urutan data dapat meningkatkan kinerja query secara signifikan.

Evaluasi optimasi juga mencakup ukuran indeks dan overhead operasi DML. Ukuran indeks menunjukkan kebutuhan ruang penyimpanan tambahan, sedangkan pengujian INSERT, UPDATE, dan DELETE digunakan untuk mengukur dampak

pemeliharaan indeks terhadap perubahan data. Hasil kedua pengujian tersebut harus dinilai berdasarkan pengukuran aktual, bukan asumsi.

Dengan demikian, pemilihan strategi optimasi harus mempertimbangkan keseimbangan antara waktu eksekusi query, biaya penyimpanan indeks, dan biaya operasi perubahan data. Keputusan akhir ditentukan berdasarkan hasil pengujian pada dataset dan lingkungan PostgreSQL yang digunakan.